Technical University of Denmark

Written examination, May 20, 2022

Course number: 02141

Aids allowed: All aid

The exam is conducted as a digital exam taken at home with the aids allowed for such exams. For this reason answers that can be obtained using a computer need to be properly explained and will not be considered acceptable otherwise. You must submit your answer as a single pdf-file; it is fine to integrate pictures of drawings into the pdf-file, but no attachments to the pdf-file will be considered. You must clearly indicate on each page of your submission which task you attempt to solve.

If you think there are mistakes in the exam set (where in a written exam you would ask to be contacted by the teacher) please send an e-mail to chmat@dtu.dk for Formal Methods and to albl@dtu.dk for Regular Languages and Context Free Languages. Any announcements resulting from this will be communicated over DTU Learn.

Exam duration: 4 hours

Weighting: 7-step scale

Exercise 1 (15%) Semantics

Consider the program graphs `PG1` - `PG3` in Figures 1 - 3 on page 3.

Question 1a: *For each of the program graphs on page 3, determine whether there exists a program in Guarded Command Language such that* $\mathbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\cdots]\!]$ *generates it. If you believe that a suitable program exists, it suffices to provide it. Otherwise, briefly justify why no such program exists.*

Question 1b: *For the program graphs* `PG2` *and* `PG3` *in Figures 2 and 3 on page 3, how many different execution sequences of the form*

$$\langle q_{\triangleright}; [\mathtt{x} \mapsto 5, \mathtt{y} \mapsto 5]\rangle \overset{\omega}{\Longrightarrow}^{*} \langle q_{\blacktriangleleft}; \sigma\rangle$$

exist for each program graph? For each such execution sequence, also provide σ*. (You do not need to write entire execution sequences.)*

Question 1c: *For each of the program graphs* `PG1` - `PG3` *in Figures 1 - 3 on page 3, determine whether it is an evolving system. Justify your answer.*

Question 1d: *An alternative definition of a deterministic system, called an* observationally deterministic system *in the following, is as follows: for all memories* $\sigma, \sigma', \sigma''$ *and all* ω, ω'*, we have*

$$\langle q_{\triangleright}; \sigma\rangle \overset{\omega}{\Longrightarrow}^{*} \langle q_{\blacktriangleleft}; \sigma'\rangle \text{ and } \langle q_{\triangleright}; \sigma\rangle \overset{\omega'}{\Longrightarrow}^{*} \langle q_{\blacktriangleleft}; \sigma''\rangle \text{ implies } \sigma' = \sigma''.$$

Give a program graph that is both an evolving system and an observationally deterministic system but not *a deterministic system in the sense of [FM]. Justify your answer.*

Question 1e: *We extend the admissible actions of the Guarded Command Language considered in [FM]: We add a new statement*

```
x := a1 DIV a2 REM y
```

to the Guarded Command Language, which performs an integer division of the value of arithmetic expression `a1` *by the value of arithmetic expression* `a2`*; the result (which is always rounded down) is assigned to variable* `x` *and the remainder is assigned to variable* `y` *afterward.*

Extend the definition of the semantics function $\mathcal{S}[\![\ldots]\!]$ *in [FM] by a new action for the statement* `x := a1 DIV a2 REM y` *from above.*

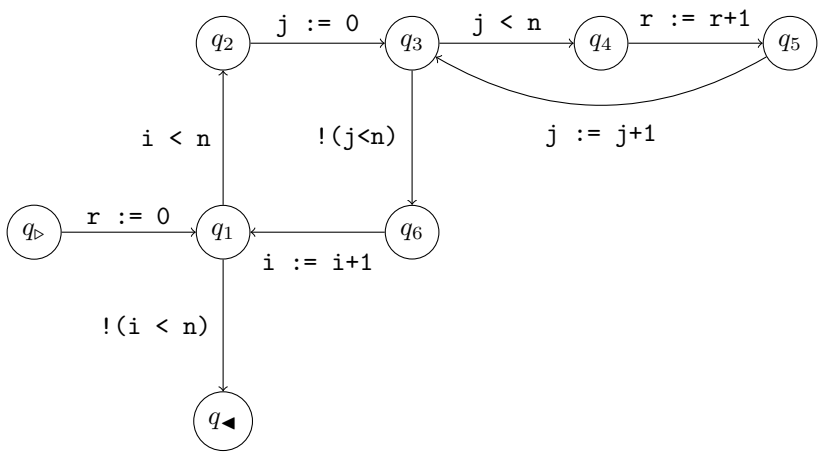


Figure 1: Program graph PG1

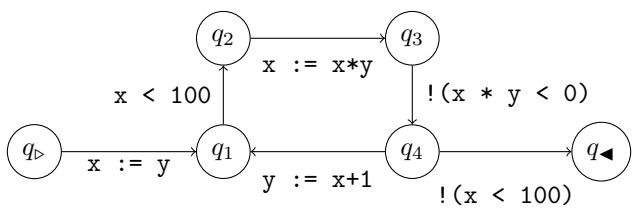


Figure 2: Program graph PG2

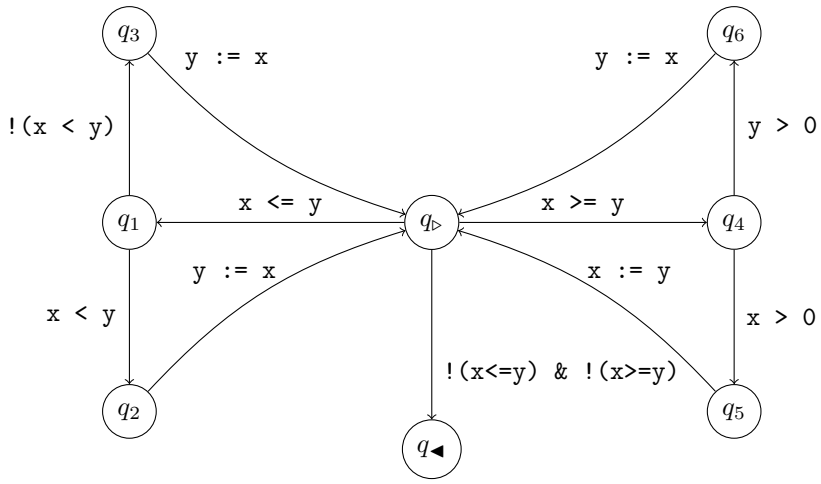
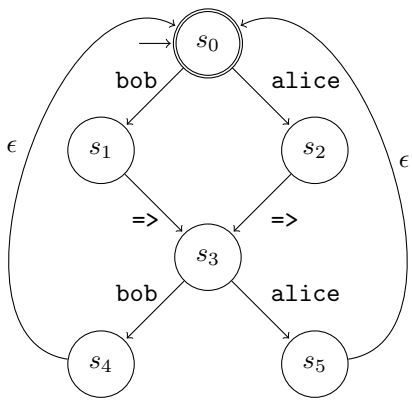


Figure 3: Program graph PG3

Exercise 2 (20%) Formal Languages

In this exercise we consider languages for describing information flows. In particular, we consider the following 3 languages:

(1) The automaton A



(2) The regular expression B

$((\texttt{alice}+\texttt{bob})\texttt{=>}(\texttt{alice}+\texttt{bob}))^+$

(3) The grammar C

$$\begin{array}{lcl} S & \rightarrow & SS \mid F \mid \epsilon \\ F & \rightarrow & \texttt{alice=>bob} \mid \texttt{bob=>alice} \end{array}$$

where the set $\{\texttt{alice}, \texttt{bob}, \texttt{=>}\}$ acts as input alphabet for A and B, and as the set of terminal symbols for C, and where the non-terminal symbols in C are S (initial) and F.

(a) Consider the strings accepted by A, B and C, i.e. their languages $\mathcal{L}(\mathsf{A})$, $\mathcal{L}(\mathsf{B})$, $\mathcal{L}(\mathsf{C})$.

Fill the table below as follows: in the cell corresponding to row x and column y write YES if the language x overlaps with language y and provide a (preferably short) string contained in both languages. Otherwise, write NO and briefly explain (in 2-3 lines) why the two languages do not overlap.

You do not need consider the diagonal.

$(\mathcal{L}(\ldots) \cap \mathcal{L}(\ldots)) \neq \emptyset$	$\mathcal{L}(\mathsf{A})$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	—		
$\mathcal{L}(\mathsf{B})$		—	
$\mathcal{L}(\mathsf{C})$			—

(b) Consider again the sets of strings accepted by A, B and C, that is, the languages $\mathcal{L}(\mathsf{A})$, $\mathcal{L}(\mathsf{B})$, $\mathcal{L}(\mathsf{C})$.

Fill the below table as follows: in the cell corresponding to row x and column y write YES and a brief explanation (in 2-3 lines) if the language x is contained in the language. Otherwise, write NO and provide a (preferably short) string that is accepted by x but not by y.

You do not need consider the diagonal.

$\mathcal{L}(\dots) \subseteq \mathcal{L}(\dots)$	$\mathcal{L}(A)$	$\mathcal{L}(\mathsf{B})$	$\mathcal{L}(\mathsf{C})$
$\mathcal{L}(\mathsf{A})$	—		
$\mathcal{L}(\mathsf{B})$		—	
$\mathcal{L}(\mathsf{C})$			—

(c) Consider the regular expression $\mathsf{D} = (\texttt{alice=>}(\texttt{alice} + \texttt{bob}))^*$. Use the transformations seen in the course to obtain an equivalent DFA. Briefly explain each step (that is, what you did and why you did it).

(d) Consider the following languages. For each language L, state whether or not L is a regular language. Briefly explain your answer in 2-3 lines.

- The language $\mathcal{L}(\mathsf{A})$, where A is described at the top of the exercise.
- The language $\mathcal{L}(\mathsf{B})$, where B is described at the top of the exercise.
- The language $\mathcal{L}(\mathsf{C})$, where C is described at the top of the exercise.
- The language L_4 of all strings accepted by A but not by C.
- The language L_5 of all strings accepted by C, where the substring `alice=>bob` appears as many times as the substring `bob=>alice`.

Exercise 3 (15%) Program Verification

Question 3a: *For each contract below (using notation as on formalmethods.dk), argue whether it is correct or not (you do not have to construct proof trees). Here,* `x`, `y`, `z` *are program variables and* `A` *and* `B` *are arrays of length* `n`.

(a)
```
{ x > 3 }
  if x >= 0 -> x := x - 3
  [] x >= 5 -> x := x + y*y
  [] x >= 7 -> x := x + y
  fi
{ x >= 0 }
```

(b)
```
{ x = 23 }
  do x > 0 -> x := x - 2
  [] x < 0 -> x := x + 4
  od
{ x = 421723 }
```

(c)
```
{0 <= x < n && 0 <= y < n && A[x] == 42 && A[y] == 17 }
  z := A[x]; A[x] := A[y]; A[y] := z
{0 <= x < n && 0 <= y < n && A[x] == 17 && A[y] == 42 }
```

(d)
```
{x = max {A[i] | 0 <= i < n } }
  x := A[y]
{A[y] = max {A[i] | 0 <= i < n } }
```

Question 3b: *Show that the contract in Figure 4 is valid with respect to partial correctness. You may either construct a proof tree (an annotated program) as in the lecture or derive and analyse shortest path fragments as in [FM].*

```
{ n >= 0 }
i := 0;
res := 0;
{ n >= 0 && i <= n && res = n * i }
do i < n ->
    j := 0;
    do j < n ->
        res := res + 1;
        j := j + 1
    od;
    i := i + 1
od
{ res = n^2 }
```

Figure 4: Contract for program C

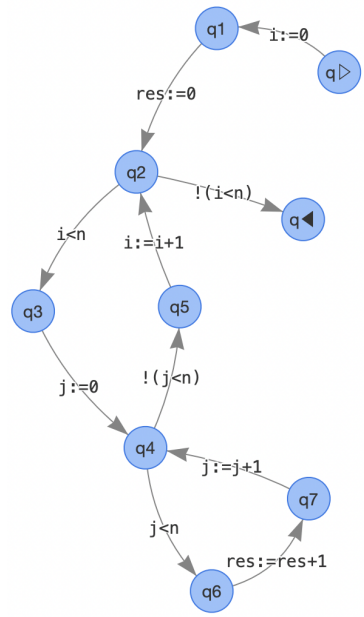


Figure 5: Program graph for C

Exercise 4 (13%) Program Analysis

Question 4a: *Explain whether the following statements about the program graphs* `PG4` *and* `PG5` *in Figures 7 - 9 (on page 14) are* `true`, `false`, *or whether no conclusive answer is possible (*`unknown`*). Your answers must be* based on the results of the detection of signs analysis *in [FM] and implemented on formalmethods.dk (that is, you may only inspect individual actions and the analysis information obtained for adjacent nodes). Explain your answer to each question.*

(a) Upon termination of `PG4`, x will be non-negative.

(b) `PG4` never divides by zero.

(c) There is at least one execution of `PG5` that terminates.

(d) In `PG5`, the sign of variable `x` in node q_1 coincides with the sign of `x` in the initial node $q_\triangleright$.

The detection of signs analysis in [FM] often loses precisions for updates involving small integers. We will thus extend it such that the numbers $1, 2$ and their negative counterparts are treated more precisely, similar to 0. That is, our refined analysis should detect whether a variable (or array element) has value $-2, -1, 0, 1, 2$, a value smaller than -2 (indicated by < -2) or a value greater than 2 (indicated by > 2).

Question 4b: *Give a formal definition of the set* $\widehat{\mathbf{Mem}}$ *of abstract memories that can be used for the refined detection of signs analysis.*

$$(\hat{\sigma}_1, \hat{\sigma}_2) \in \widehat{\mathbf{Mem}} \;=$$

Question 4c: *Apply the refined detection of signs analysis to compute an analysis assignment for the program graph in Figure 7 on page 14. You may assume that all program variables are initially larger than 2. Briefly explain your answer, particularly when performing a division operation.*

$$\begin{aligned}
\mathbf{A}(q_\triangleright) &= \{\, [\mathtt{x} \mapsto > 2,\ \mathtt{y} \mapsto > 2] \,\} \\
\mathbf{A}(q_1) &= \\
\mathbf{A}(q_3) &= \\
\mathbf{A}(q_4) &= \\
\mathbf{A}(q_2) &= \\
\mathbf{A}(q_\blacktriangleleft) &=
\end{aligned}$$

```
x := 2;
if
   true -> y := -2/x
[] true -> y := x/3
fi;
x := y / -x
```

Figure 6: Code for PG4

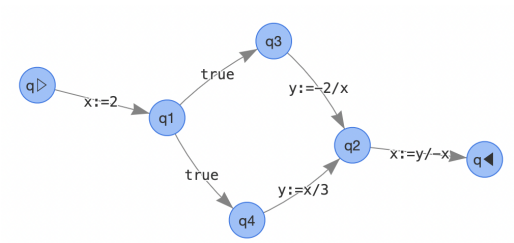


Figure 7: Program graph PG4

```
y := x;
do
  y > 0 -> x := x - 1;
           y := y - x;
           x := x +1
[]
  y < 0 -> x := x + 1;
           y := y + x;
           x := x - 1
od
```

Figure 8: Code for PG5

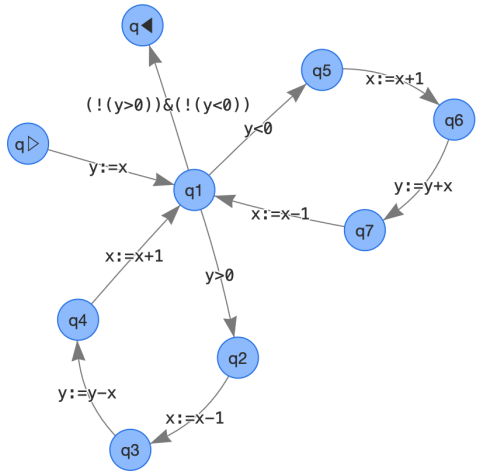


Figure 9: Program graph PG5

Exercise 5 (7%) Information Flow

Let u and v be the only two variables in a GCL program and let us consider its deterministic program graph with initial state $q_{\triangleright}$ and final state $q_{\blacktriangleleft}$. We say that *"u interferes with v"* if we can find two memories σ_1 and σ_2 such that:

(i) $\sigma_1(v) = \sigma_2(v)$

(ii) $\langle q_{\triangleright}, \sigma_1 \rangle \Rightarrow^* \langle q_{\blacktriangleleft}, \sigma_1' \rangle$

(iii) $\langle q_{\triangleright}, \sigma_2 \rangle \Rightarrow^* \langle q_{\blacktriangleleft}, \sigma_2' \rangle$

(iv) $\sigma_1'(v) \neq \sigma_2'(v)$

NOTE: although u does not explicitly appear in the definition above, it is of course in the domain of the memories $\sigma_1, \sigma_2, \ldots$.

Consider the following GCL programs:

(a) `x := y`

(b) `y := x ; x := 0`

(c) `if y>0 -> y:=y-1 [] true -> x:=1 fi`

(d) `if x=0 -> y:=x [] true -> y:=0 fi`

For each program

- Consider a confidentiality policy $\mathsf{public} < \mathsf{private}$ and specify a security assignment for `x` and `y` such that the program is not secure w.r.t to the notion of security in [FM]. Explain why the program is not secure in 2-3 lines.
- Given the security assignment, determine if the private variable (if any) interferes with the public variable (if any). If you state that there is an interference, then provide memories σ_1 and σ_2 with the above stated properties (i–iv). Otherwise, explain why such pair of memories do not exist.

Exercise 6 (20%) Context-free Languages

The following context-free grammar provides a syntax for a new language that we call *Guarded Command Modelling Language (GCML)*:

$$
\begin{array}{lcll}
C & \rightarrow & A & \text{(actions)} \\
 & | & C\,\texttt{;}\,C & \text{(sequencing)} \\
 & | & C\,\texttt{[]}\,C & \text{(branching)} \\
 & | & C\,\texttt{!} & \text{(looping)} \\
 & | & \texttt{[}C\texttt{]} & \text{(grouping)} \\
A & \rightarrow & \texttt{skip} \mid \texttt{assign} &
\end{array}
$$

where the set of non-terminal symbols is $N = \{C, A\}$, the set of terminal symbols is $T = \{\texttt{skip}, \texttt{assign}, \texttt{;}, \texttt{[}, \texttt{]}, \texttt{!}\}$ and the initial symbol is C. We call a string accepted by the grammar a "model".

(a) Design a set of datatypes that are suitable to store abstract representations (ASTs) of models and show how the model

```
[ assign ; [assign [] skip ] ] !
```

would be represented as a value of your datatype.

(b) Consider the following operators for composing models: `_;_`, `_[]_`, `_!` and `[_]`. Fill the table below as follows: in the cell corresponding to row x and column y write YES if, according to the above grammar (without any additional operator precedence), the operators x and y can yield an ambiguity. Otherwise, leave the cell blank.

	`_;_`	`_[]_`	`_!`	`[_]`
`_;_`				
`_[]_`				
`_!`				
`[_]`				

(c) If you have discovered at least one ambiguity in (b), showcase it by providing an example of a model that has at least two distinct parse trees. Otherwise, explain in 2-3 lines why no string/model exists that can have two parse trees.

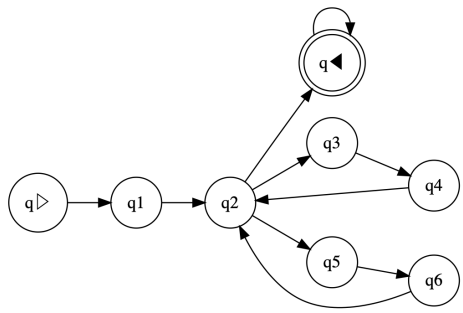
(d) If you have discovered at least one ambiguity in (b), provide change the grammar to remove the ambiguity. You do not need to solve all ambiguities you have discovered. Explain in 2-3 lines the change that you have done and why it solves the ambiguity.

(e) Construct a Pushdown Automaton (PDA) that accepts the same language as the grammar described at the beginning of the exercise, and show how it runs on the string `[skip]` by providing an accepting sequence of configurations (aka instantaneous descriptions). How many steps does your PDA take to recognize the string?

(f) Can the string discussed in (e) be recognized in less steps by another (equivalent) PDA? If no, explain why in 2-3 lines. If yes, provide the new PDA, show how it would recognize the string in less steps, and explain in 2-3 lines the idea behind the new PDA.

Exercise 7 (10%) Model Checking

Consider the following transition system:



where

- the set of initial states contains only the state $q_{\triangleright}$;
- the set of atomic propositions are the names of the states, and
- the labelling function for states therefore trivially maps a state to the set consisting of just that state.

Recall that a state formula Φ of CTL holds on a transition system if and only if it holds on all initial states, which in this case is merely the initial node $q_{\triangleright}$.

Next consider the following CTL formulae :

1 $\mathbf{EF}\, q_{\blacktriangleleft}$

2 $\mathbf{AF}\, q_{\blacktriangleleft}$

3 $\mathbf{AGEF}\, q_{\blacktriangleleft}$

4 $\mathbf{EF\,AG} q_{\blacktriangleleft}$

5 $\mathbf{E}(\neg q_2)\mathbf{U} q_{\blacktriangleleft}$

6 $\mathbf{A}(\neg q_2)\mathbf{U} q_{\blacktriangleleft}$

7 $\mathbf{E}(\neg q_4)\mathbf{U} q_{\blacktriangleleft}$

8 $\mathbf{A}(\neg q_4)\mathbf{U} q_{\blacktriangleleft}$

Which of the above formulae hold on the transition system? Explain your answer in 1-2 lines.